DEQUE DOUBLE-ENDED QUEUE.

ESTRUTURA DE DADOS E APLICAÇÃO EM UM SIMULADOR DE FILA DE ATENDIMENTO

Integrantes: Eduarda e Anderson

O que é Deque? Double-Ended Queue

- deque significa Double-Ended Queue
- É uma estrutura de dados do módulo collections do Python
- Permite inserir e remover elementos nas duas extremidades
- Pode ser utilizada para implementar: Filas, Pilhas, Sistemas de prioridade e Processamento de elementos nas extremidades

Por que utilizar Deque?

Fila Tradicional

ENTRADA → [A] [B] [C] [D] → SAÍDA
- Inserção apenas em um lado
- Remoção apenas no outro lado
- Operações limitadas a uma extremidade
- Não permite manipulação no início
- Exemplo: list.append() e list.pop(0) → O(n)

Deque

← [A] [B] [C] [D] →
INÍCIO FIM
- Inserção e remoção em ambos os lados
- appendleft() e popleft() no INÍCIO
- append() e pop() no FIM
- Principal vantagem: manipulação eficiente das duas extremidades em O(1)

Como utilizar no Python?

- Importação:

from collections import deque

- Criação de um deque vazio:

fila = deque()

- Iniciar com elementos:

fila = deque(['A', 'B', 'C'])

- O Python disponibiliza essa implementação através do módulo collections — não é necessário implementar do zero.

As 4 Operações Fundamentais

append() / appendleft()

Adiciona elementos na fila:
fila.append("A")
→ adiciona no FINAL
fila.appendleft("B")
→ adiciona no INÍCIO
Complexidade: O(1)

pop() / popleft()

Remove elementos da fila:
fila.pop()
→ remove do FINAL
fila.popleft()
→ remove do INÍCIO
Complexidade: O(1)

Diagrama das Operações

appendleft() →→ [A | B | C | D] ←← append()
popleft() ←← [A | B | C | D] →→ pop()
Operações nas extremidades são sempre O(1) — independente do tamanho da fila.

Outras operações da API

Métodos disponíveis

- extend(["A","B","C"]) — adiciona vários no final
- extendleft([...]) — adiciona vários no início
- clear() — remove todos os elementos
- copy() — cria uma cópia superficial
- count(x) — conta ocorrências de x
- index(x) — retorna índice de x

rotate() e reverse()

- rotate(1): desloca elementos para a direita

Antes: A → B → C → D

Depois: D → A → B → C

- rotate(-1): desloca para a esquerda
- reverse(): inverte a ordem dos elementos

Acesso aos Elementos

Acesso por Índice

fila[0] → primeiro elemento
fila[-1] → último elemento
⚠️ Não é otimizado para acesso frequente a posições intermediárias.
O deque permite acesso por índice, mas sua principal vantagem está nas operações nas extremidades.

Comparação: List vs Deque

- Acesso por índice → LIST ✓
- Operações nas pontas → DEQUE ✓
- Use LIST quando precisar acessar elementos por posição com frequência.
- Use DEQUE quando as operações forem concentradas nas extremidades da estrutura.

Arquitetura e funcionamento interno

Como o deque funciona internamente?
No CPython, o deque é implementado com blocos de memória duplamente ligados:
• Estrutura interna: INÍCIO → [BLOCO] ⇄ [BLOCO] ⇄ [BLOCO] ← FIM
• appendleft() / popleft() operam no INÍCIO
• append() / pop() operam no FIM
• Operações nas extremidades NÃO deslocam todos os elementos
• Cada bloco armazena múltiplos elementos contíguos
• Novos blocos são alocados/desalocados nas pontas conforme necessário
⚡ Essa arquitetura garante O(1) nas operações das extremidades.

Big O
Complexidade das Operações

- Big O representa o crescimento do custo conforme aumenta o número de elementos.
- O(1) — Constante:

append() · appendleft()
pop() · popleft()

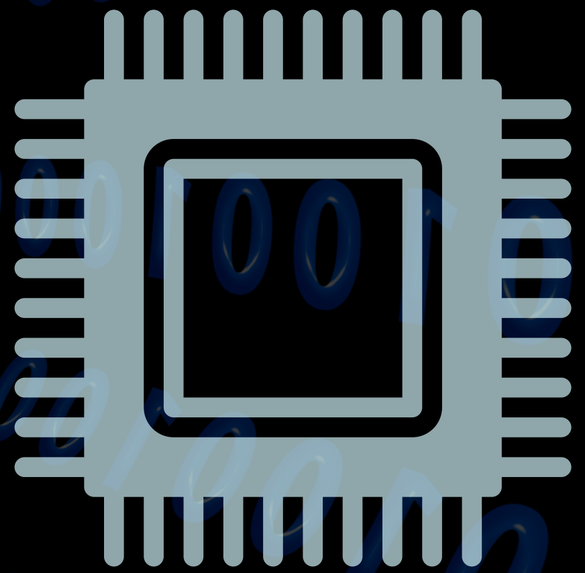
- O(k) — Linear no argumento:

extend() · extendleft()

- O(n) — Linear nos elementos:

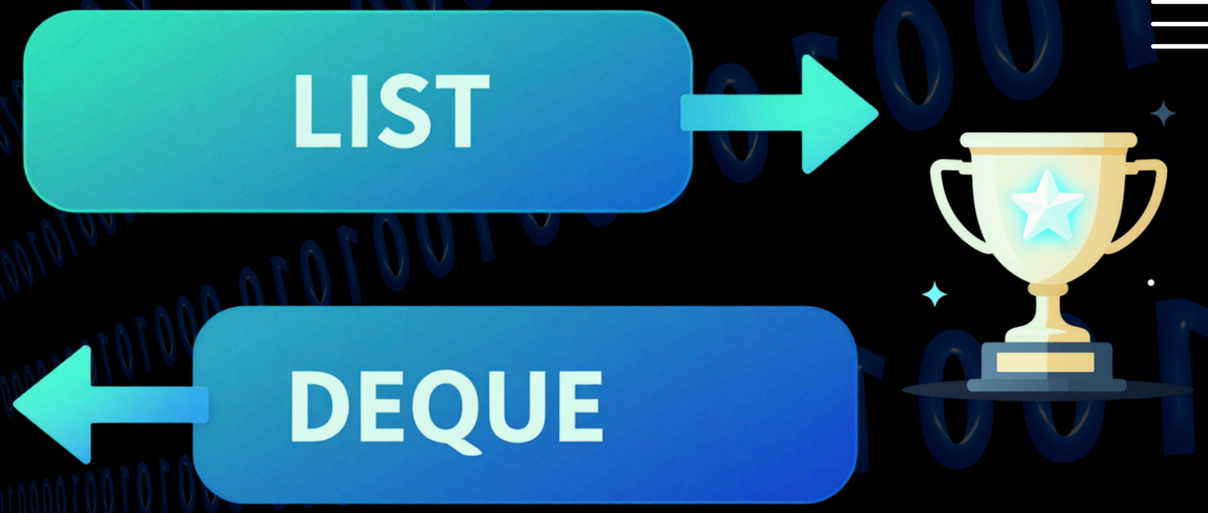
count() · index() · clear()

- Operações nas extremidades são O(1)



Deque
List

×



Quando usar List?

✅ Acesso por índice: O(1)
✅ Inserção no final: O(1)*
✅ Remoção no final: O(1)
❌ Inserção no início: O(n)
❌ Remoção no início: O(n)
❌ Duas extremidades: —
exemplo:
lista.pop(0) → O(n)

Quando usar Deque?

⭐ Acesso por índice: O(n)
✅ Inserção no final: O(1)
✅ Remoção no final: O(1)
✅ Inserção no início: O(1)
✅ Remoção no início: O(1)
✅ Duas extremidades: Otimizado
exemplo:
fila.popleft() → O(1)

RIMBERIO

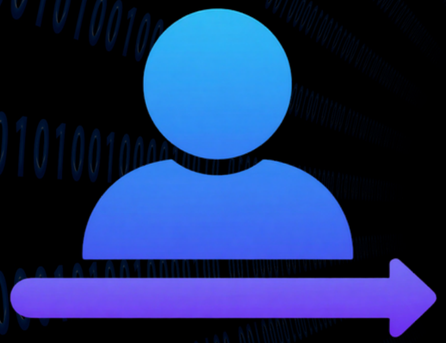
Deque como Fila

FIFO — First In, First Out
Primeiro a entrar → primeiro a sair

```
fila = deque()
fila.append("A")
fila.append("B")
fila.append("C")
fila.popleft()  # remove "A"
```

Antes: A → B → C
Depois: B → C

Aplicação: Simulador de Atendimento



🧍 Cliente Normal
fila.append(cliente)
Entra no final da fila.
FLUXO: NORMAL → append() → [A][B][C][D] [NOVO]

⭐ Cliente Prioritário
fila.appendleft(cliente)
Entra no início da fila.
FLUXO: PRIORIDADE → appendleft() → [P] [A][B][C][D]

✅ Atendimento
client = fila.popleft()
Remove o próximo cliente.
FLUXO: ATENDIMENTO → popleft() → [B][C] [D]